

Qwen3-Next: Hacia la máxima eficiencia en entrenamiento e inferencia

10/09/2025 · 54 minutos · 10813 palabras · QwenTeam · Traducciones: Chino simplificado

Qwen3-Next



Introducción

Creemos que el **escalado de longitud de contexto** y el **escalado de parámetros totales** son dos tendencias importantes en el futuro de los modelos grandes. Para mejorar aún más la eficiencia del entrenamiento y la inferencia en entornos de contexto extenso y con muchos parámetros, diseñamos una arquitectura de modelo completamente nueva llamada Qwen3-Next. En comparación con la estructura MoE de Qwen3, Qwen3-Next introduce varias mejoras clave: un mecanismo de atención híbrido, una estructura de mezcla de expertos (MoE) altamente dispersa, optimizaciones que favorecen la estabilidad del entrenamiento y un mecanismo de predicción de múltiples tokens para una inferencia más rápida.

Basándonos en esta nueva arquitectura, entrenamos el modelo **Qwen3-Next-80B-A3B-Base**, un modelo de 80 mil millones de parámetros que activa solo 3 mil millones durante la inferencia. Este modelo base logra un rendimiento comparable (o incluso ligeramente superior) al del modelo denso Qwen3-32B,

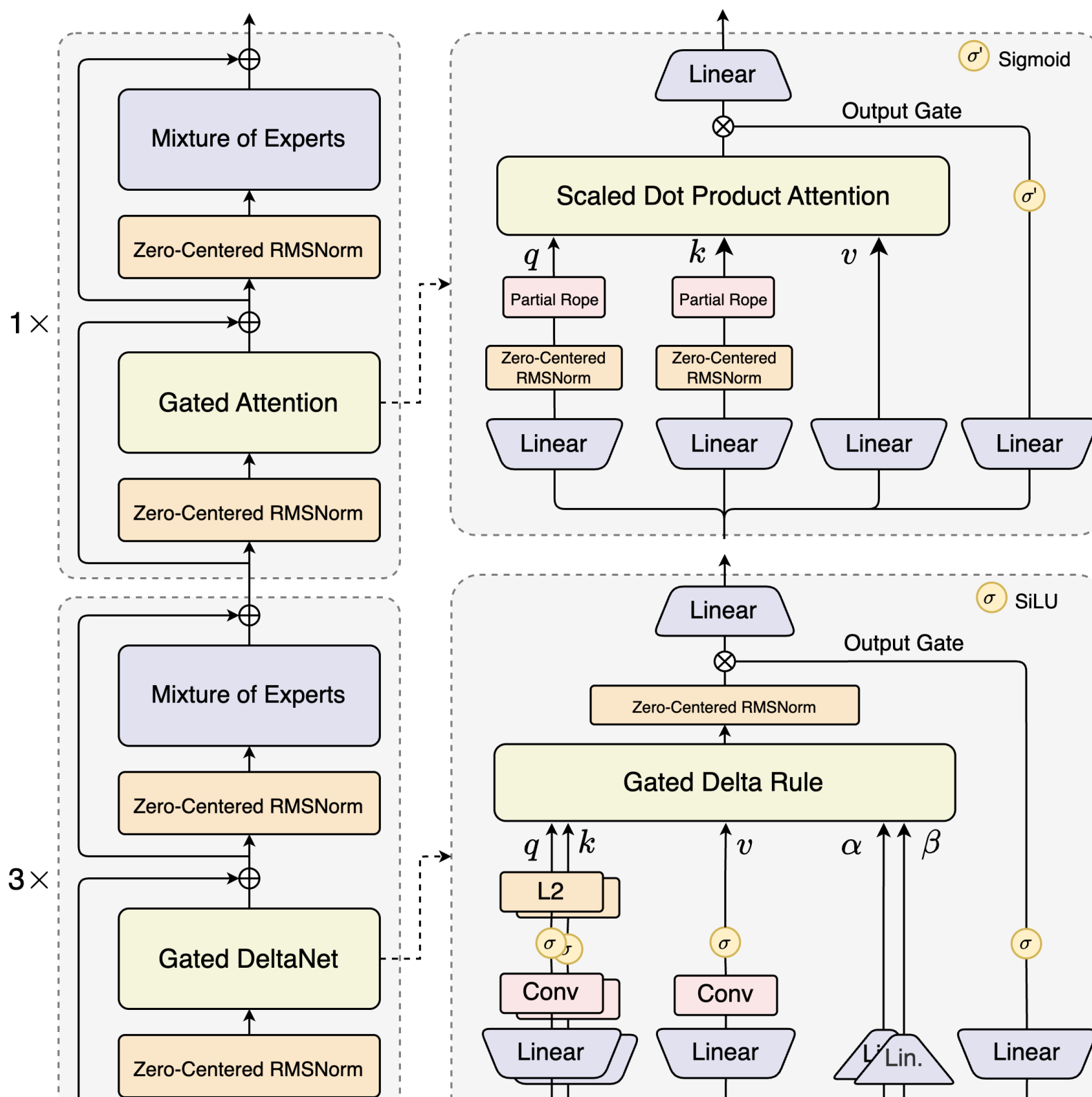
utilizando menos del 10 % de su coste de entrenamiento (horas de GPU). En la inferencia, especialmente con longitudes de contexto superiores a 32 000 tokens, ofrece un rendimiento más de 10 veces superior, **logrando una eficiencia extrema tanto en el entrenamiento como en la inferencia** .

Desarrollamos y publicamos dos versiones preentrenadas basadas en Qwen3-Next-80B-A3B-Base: **Qwen3-Next-80B-A3B-Instruct** y **Qwen3-Next-80B-A3B-Thinking** . Resolvemos los problemas de estabilidad y eficiencia que persistían en el entrenamiento de aprendizaje por refuerzo (RL), causados por la arquitectura híbrida de atención + MoE de alta dispersión. Esto resultó en mejoras tanto en la velocidad de entrenamiento de RL como en el rendimiento final.

El **Qwen3-Next-80B-A3B-Instruct** tiene un rendimiento comparable al de nuestro modelo insignia **Qwen3-235B-A22B-Instruct-2507** y muestra claras ventajas en tareas que requieren un contexto ultralargo (**hasta 256K tokens**). El **Qwen3-Next-80B-A3B-Thinking** sobresale en tareas de razonamiento complejas, superando a modelos de mayor costo como **Qwen3-30B-A3B-Thinking-2507** y **Qwen3-32B-Thinking** , **superando al Gemini-2.5-Flash-Thinking** de código cerrado en múltiples pruebas de rendimiento y acercándose al rendimiento de nuestro modelo de nivel superior **Qwen3-235B-A22B-Thinking-2507** .

Ya hemos lanzado Qwen3-Next en [Hugging Face](#) y [ModelScope](#) . Además, todos pueden usar el servicio Qwen3-Next a través de Alibaba Cloud Model Studio y [el catálogo de API de NVIDIA](#) .

Características principales





Arquitectura híbrida: DeltaNet con compuertas + Atención con compuertas

La atención lineal rompe la complejidad cuadrática de la atención estándar y es más eficiente para contextos largos. Sin embargo, observamos que usar solo atención lineal o solo atención estándar tiene limitaciones: la atención lineal es rápida pero débil en la recuperación; la atención estándar es costosa y lenta durante la inferencia.

Mediante experimentos sistemáticos, encontramos que Gated DeltaNet [1] ofrece una mayor capacidad de aprendizaje en contexto que métodos comúnmente utilizados como Sliding Window Attention o Mamba2. Cuando combinamos Gated DeltaNet con atención estándar en una proporción de 3:1 (el 75 % de las capas usan Gated DeltaNet y el 25 % mantienen la atención estándar), el modelo supera consistentemente a cualquier arquitectura monolítica, logrando un mejor rendimiento y una mayor eficiencia.

En las capas de atención estándar, agregamos varias mejoras:

1. Adopte el mecanismo de compuerta de salida de nuestro trabajo anterior [2] para reducir los problemas de bajo rango en la atención.
2. Aumentar la dimensión por cabezal de atención de 128 a 256.
3. Aplique la codificación de posición rotativa solo al primer 25 % de las dimensiones de posición, lo que mejora la extrapolación a secuencias más largas.

MoE ultradisperso: Activación de solo el 3,7% de los parámetros

Qwen3-Next utiliza un diseño MoE altamente disperso: 80 mil millones de parámetros en total, pero solo ~3 mil millones activados por paso de inferencia. Los experimentos muestran que, con el balanceo de carga global [4], aumentar el total de parámetros expertos manteniendo fijos los expertos activados reduce de manera constante la pérdida de entrenamiento.

En comparación con el MoE de Qwen3 (128 expertos en total, 8 enrutados), Qwen3-Next se expande a 512 expertos en total, combinando 10 expertos enrutados + 1 experto compartido, lo que maximiza el uso de recursos sin perjudicar el rendimiento.

Diseños que favorecen la estabilidad durante el entrenamiento

Descubrimos que el mecanismo de control de la salida de atención ayuda a eliminar problemas como el sumidero de atención [5] y la activación masiva [6], lo que garantiza la estabilidad numérica en todo el modelo.

En Qwen3, utilizamos QK-Norm, pero observamos que algunos pesos de norma de las capas se vuelven anormalmente grandes. Para solucionar esto y mejorar aún más la estabilidad, Qwen3-Next adopta Zero-Centered RMSNorm [7] y aplica una disminución gradual de los pesos de norma para evitar un crecimiento ilimitado.

También normalizamos los parámetros del enrutador MoE durante la inicialización [8], asegurando que cada experto sea seleccionado sin sesgos al principio del entrenamiento, lo que reduce el ruido de la inicialización aleatoria.

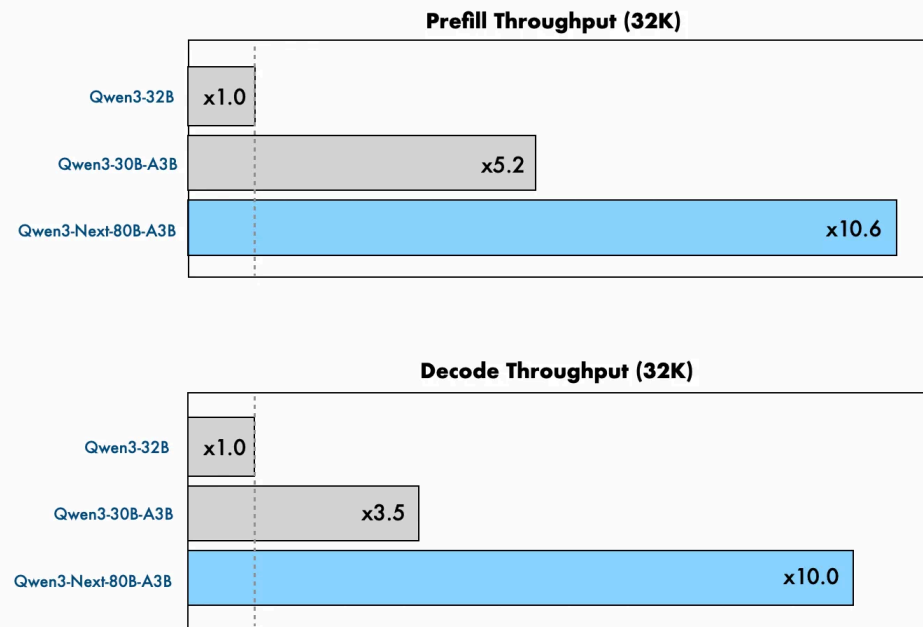
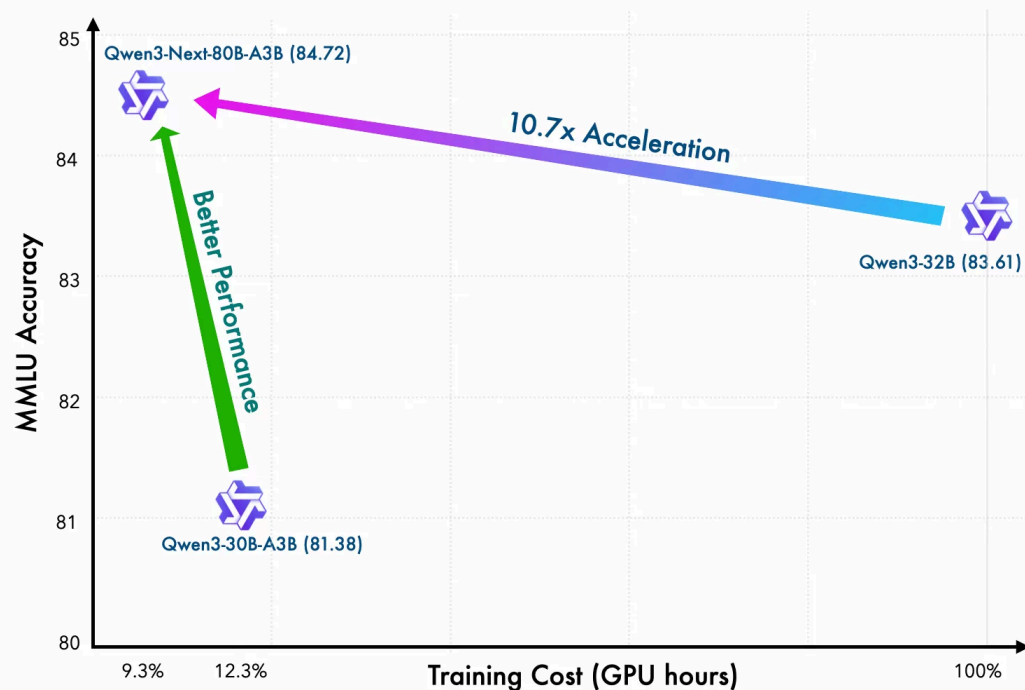
Estos diseños centrados en la estabilidad hacen que los experimentos a pequeña escala sean más fiables y ayudan a que el entrenamiento a gran escala se desarrolle sin problemas.

Predicción de múltiples tokens

Qwen3-Next introduce un mecanismo nativo de predicción de múltiples tokens (MTP) [3][9][10], que no solo produce un módulo MTP con una alta tasa de aceptación para la decodificación especulativa, sino que también mejora el rendimiento general. Además, Qwen3-Next optimiza específicamente el rendimiento de inferencia de múltiples pasos de MTP, mejorando aún más la tasa de aceptación de la decodificación especulativa en escenarios reales a través de un entrenamiento de múltiples pasos que mantiene la coherencia entre el entrenamiento y la inferencia.

Entrenamiento previo

Eficiencia del preentrenamiento y velocidad de inferencia

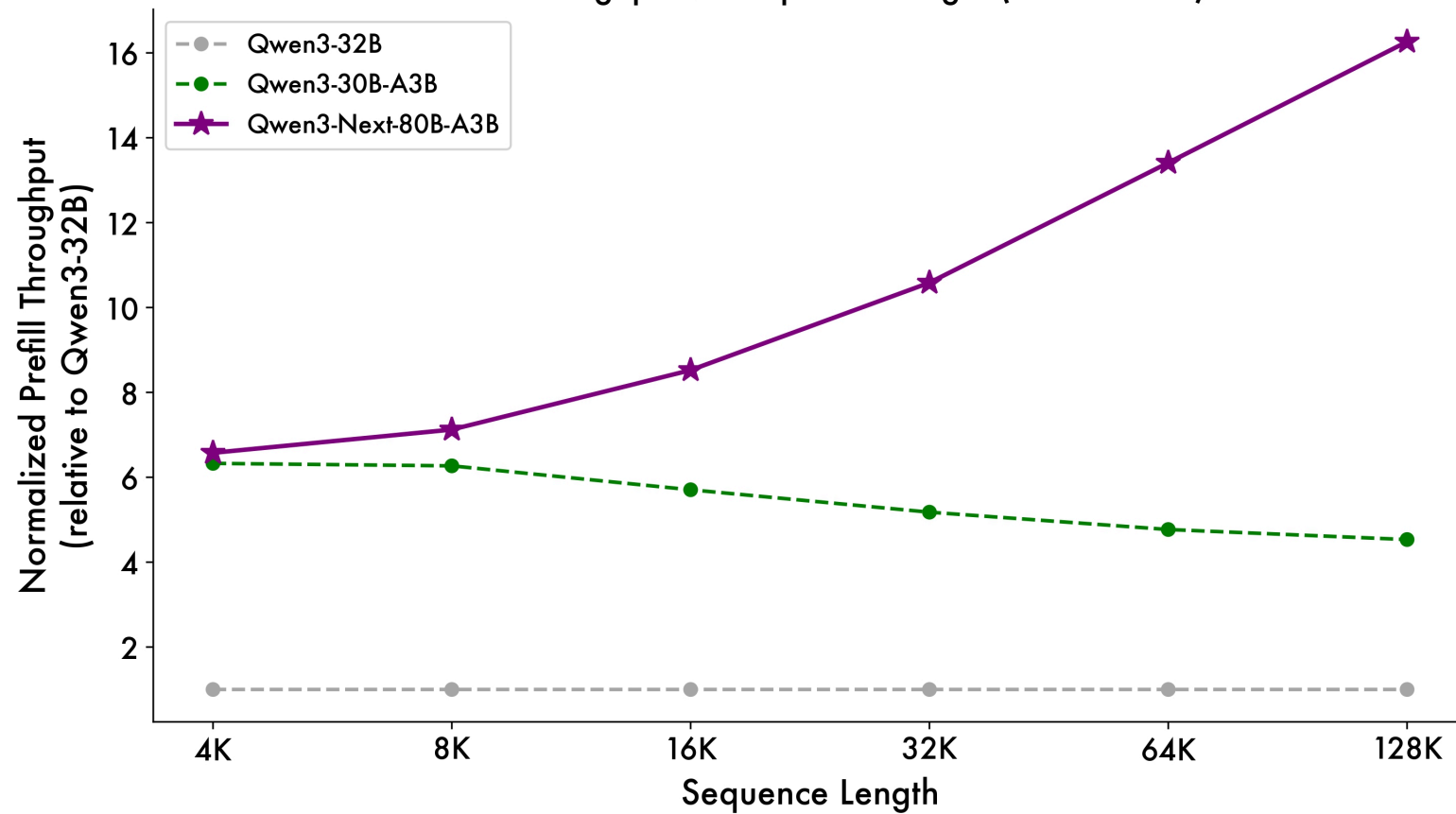


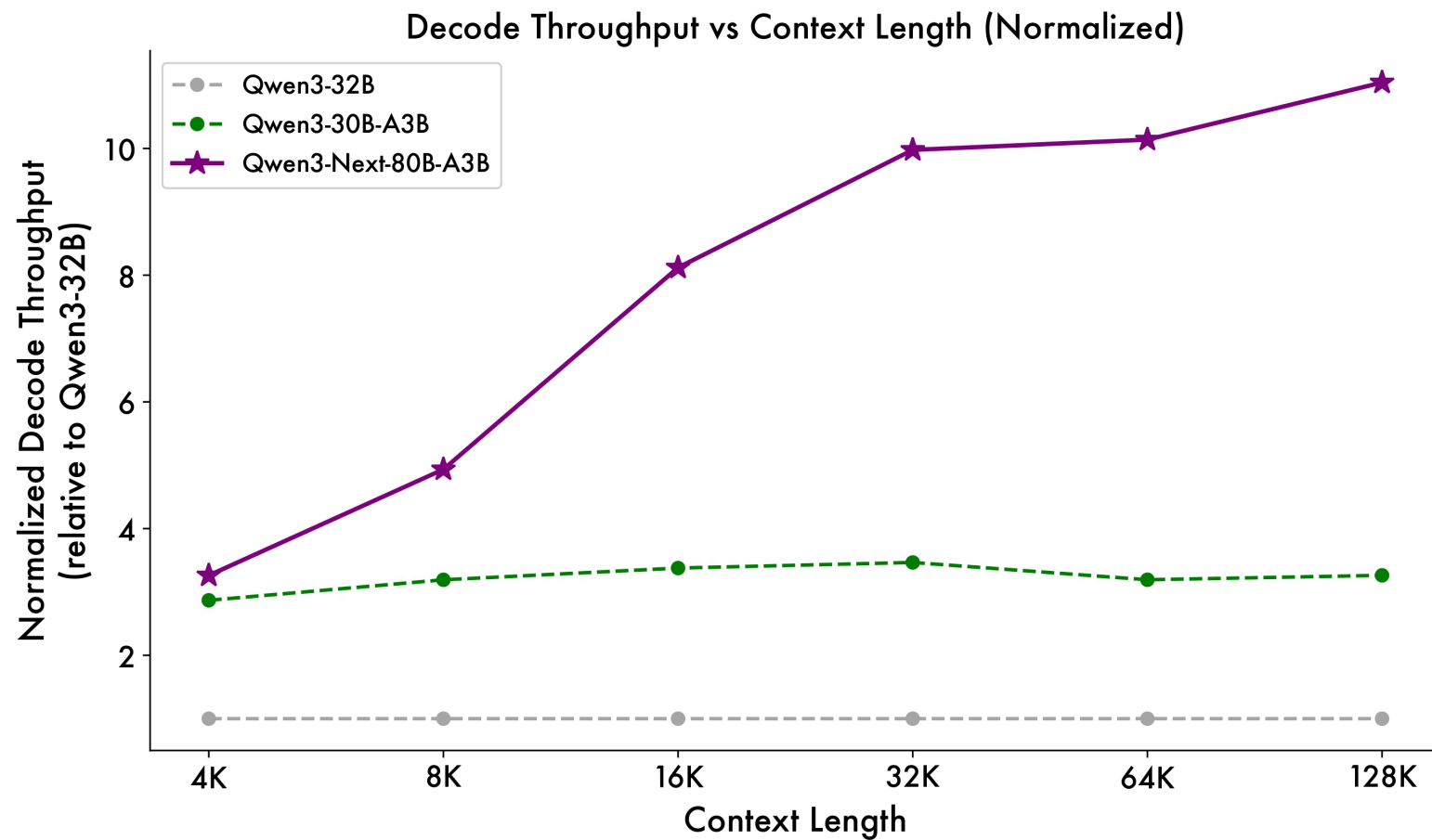
Qwen3-Next se entrena con un subconjunto muestreado uniformemente (15T tokens) del corpus de preentrenamiento de Qwen3, que consta de 36T tokens. Utiliza menos del 80 % de las horas de GPU necesarias para Qwen3-30A-3B y solo el 9,3 % del coste computacional de Qwen3-32B, logrando un rendimiento superior. Esto demuestra una eficiencia y un valor de entrenamiento excepcionales.

Gracias a su arquitectura híbrida, Qwen3-Next también destaca en inferencia:

- Etapa de prellenado** : Con una longitud de contexto de 4K, el rendimiento es casi 7 veces superior al de Qwen3-32B. Más allá de los 32K, es más de 10 veces más rápido.
- Etapa de decodificación** : Con un contexto de 4K, el rendimiento es casi 4 veces mayor. Incluso con más de 32K, mantiene una ventaja de velocidad de más de 10 veces.

Prefill Throughput vs Sequence Length (Normalized)





Rendimiento del modelo base

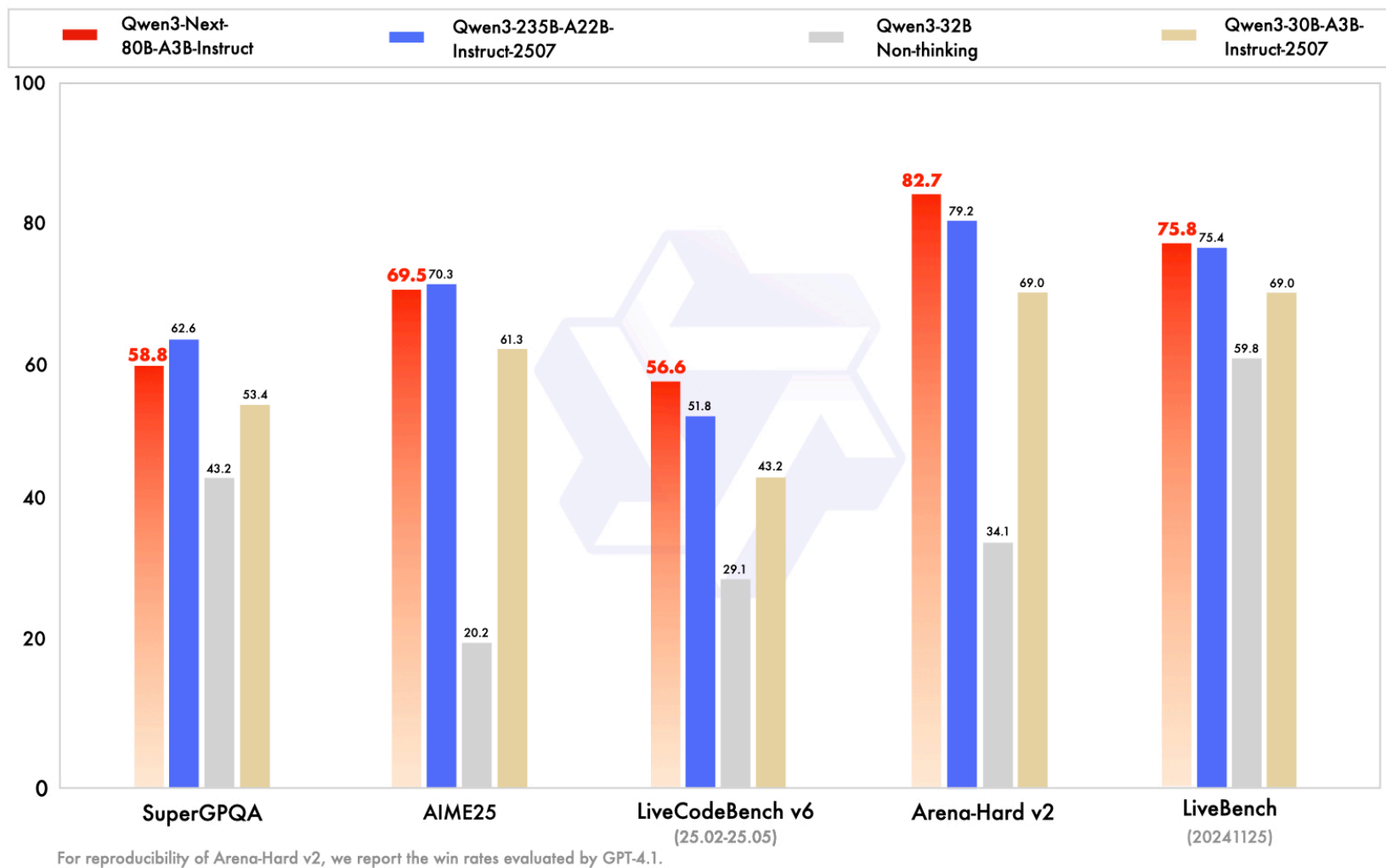
Qwen3-Next-80B-A3B-Base activa solo una décima parte de los parámetros que no son de incrustación que utiliza Qwen3-32B-Base, pero lo supera en la mayoría de las pruebas de rendimiento y supera significativamente a Qwen3-30B-A3B. Esto demuestra una eficiencia excepcional y un rendimiento sólido.

	Qwen3-30B-A3B	Qwen3-32B	Qwen3-Next-80B-A3B	Qwen3-235B-A22B
	Base	Base	Base	Base
Architecture	MoE	Dense	MoE	MoE
# Total Params	30B	32B	80B	235B
# Activated Params	3B	32B	3B	22B
<i>General Tasks</i>				
MMLU	81.38	83.61	<u>84.72</u>	87.81
MMLU-Redux	81.17	83.41	<u>83.80</u>	87.40
MMLU-Pro	61.49	65.54	<u>66.05</u>	68.18
SuperGPQA	35.72	39.78	<u>41.52</u>	44.06
BBH	81.54	<u>87.38</u>	87.13	88.87
<i>Math, STEM & Coding Tasks</i>				
GPQA	43.94	49.49	43.43	<u>47.47</u>
GSM8K	91.81	<u>93.40</u>	90.30	94.39
MATH	59.04	61.62	<u>62.36</u>	71.84
EvalPlus	71.45	72.05	<u>72.89</u>	77.60
CRUX-O	67.20	72.50	<u>74.25</u>	79.00
<i>Multilingual Tasks</i>				
MGSM	79.11	<u>83.06</u>	81.28	83.53
MMMLU	81.46	83.83	<u>84.43</u>	86.70
INCLUDE	67.00	67.87	<u>69.79</u>	73.46

Después del entrenamiento

Instruir el rendimiento del modelo

Qwen3-Next-80B-A3B-Instruct supera significativamente a Qwen3-30B-A3B-Instruct-2507 y Qwen3-32B-Non-thinking, y logra resultados que casi igualan a nuestro producto estrella, Qwen3-235B-A22B-Instruct-2507.

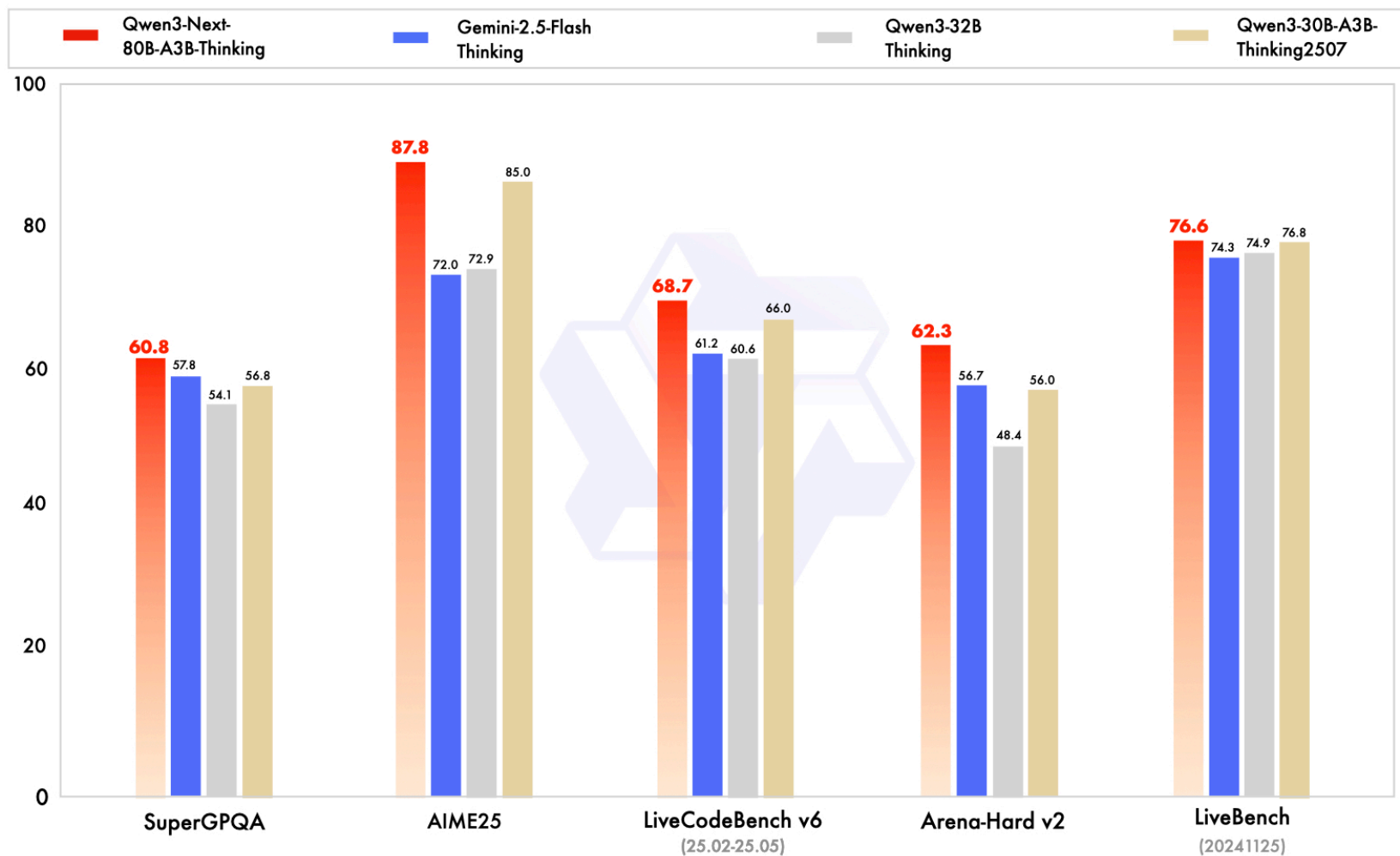


En RULER, Qwen3-Next-80B-A3B-Instruct supera a Qwen3-30B-A3B-Instruct-2507 (que tiene más capas de atención) en todas las longitudes, e incluso supera a Qwen3-235B-A22B-Instruct-2507 (que tiene más capas en general) en un contexto de 256K. Esto demuestra la solidez del diseño híbrido Gated DeltaNet + Gated Attention para tareas de contexto extenso.

Model	RULER															
	Avg.	4K	8K	16K	32K	64K	96k	128K	192k	256k	384k	512k	640k	768k	896k	1M
Qwen3-30B-A3B-Instruct-2507	86.8	98.0	96.7	96.9	97.2	93.4	91.0	89.1	89.8	82.5	83.6	78.4	79.7	77.6	75.7	72.8
Qwen3-235B-A22B-Instruct-2507	92.5	98.5	97.6	96.9	97.3	95.8	94.9	93.9	94.5	91.0	92.2	90.9	87.8	84.8	86.5	84.5
Qwen3-Next-80B-A3B-Instruct	91.8	98.5	99.0	98.0	98.7	97.6	95.0	96.0	94.0	93.5	91.7	86.9	85.5	81.7	80.3	80.3

Rendimiento del modelo de pensamiento

El Qwen3-Next-80B-A3B-Thinking supera a modelos de mayor costo como el Qwen3-30B-A3B-Thinking-2507 y el Qwen3-32B-Thinking. Supera al Gemini-2.5-Flash-Thinking de código cerrado en múltiples pruebas de rendimiento y se acerca a nuestro último modelo insignia, el Qwen3-235B-A22B-Thinking-2507, en métricas clave.



Desarrollar con Qwen3

Todos los ejemplos que aparecen a continuación se basan en la versión Qwen3-Next-80B-A3B-Instruct. Para el modelo Thinking, consulte <https://huggingface.co/Qwen/Qwen3-Next-80B-A3B-Thinking> .

Transformers con caras abrazando

El código de Qwen3-Next se ha fusionado con la rama principal de Hugging Face transformers.

caparazón

```
1 pip install git+https://github.com/huggingface/transformers.git@main
```

Con versiones anteriores, se encontrará con el siguiente error:

```
1 KeyError: 'qwen3_next'
```

A continuación se incluye un fragmento de código que ilustra cómo utilizar el modelo para generar contenido a partir de los datos de entrada proporcionados.

pitón

```
1 from transformers import AutoModelForCausalLM, AutoTokenizer
2
3 model_name = "Qwen/Qwen3-Next-80B-A3B-Instruct"
4
5 # load the tokenizer and the model
6 tokenizer = AutoTokenizer.from_pretrained(model_name)
7 model = AutoModelForCausalLM.from_pretrained(
8     model_name,
9     dtype="auto",
10    device_map="auto",
11 )
12
13 # prepare the model input
```

```

14 prompt = "Give me a short introduction to large language model."
15 messages = [
16     {"role": "user", "content": prompt},
17 ]
18 text = tokenizer.apply_chat_template(
19     messages,
20     tokenize=False,
21     add_generation_prompt=True,
22 )
23 model_inputs = tokenizer([text], return_tensors="pt").to(model.device)
24
25 # conduct text completion
26 generated_ids = model.generate(
27     **model_inputs,
28     max_new_tokens=16384,
29 )
30 output_ids = generated_ids[0][len(model_inputs.input_ids[0]):].tolist()
31
32 content = tokenizer.decode(output_ids, skip_special_tokens=True)
33
34 print("content:", content)

```

Nota :

La predicción de múltiples tokens (MTP) no está disponible generalmente en Hugging Face Transformers.

Nota :

La mejora en la eficiencia o el rendimiento depende en gran medida de la implementación.

Se recomienda adoptar un marco de inferencia específico, por ejemplo, SGLang y vLLM, para las tareas de inferencia.

Consejo

Dependiendo de la configuración de inferencia, puede observar una mayor eficiencia con [flash-linear-attention](#) y [causal-conv1d](#). Consulte los enlaces anteriores para obtener instrucciones y requisitos detallados.

Para la implementación, puede utilizar la última versión `sglang` vllm crear un punto final de API compatible con OpenAI.

SGLang

[SGLang](#) es un marco de trabajo de servicio rápido para modelos de lenguaje y modelos de lenguaje de visión de gran tamaño. SGLang puede utilizarse para lanzar un servidor con un servicio API compatible con OpenAI.

SGLang ha dado soporte a Qwen3-Next en su mainrama, que se puede instalar desde el código fuente:

caparazón

```
1 pip install 'sglang[all] @ git+https://github.com/sgl-project/sglang.git@main#subdirectory=python'
```

El siguiente comando se puede utilizar para crear un punto final de API `http://localhost:30000/v1` con una longitud de contexto máxima de 256K tokens utilizando paralelismo tensorial en 4 GPU.

caparazón

```
1 SGLANG_ALLOW_OVERWRITE_LONGER_CONTEXT_LEN=1 python -m sglang.launch_server --model-path Qwen/Qwen3-Next-80B-A
```

Se recomienda el siguiente comando para MTP con el resto de la configuración igual que la anterior:

caparazón

```
1 SGLANG_ALLOW_OVERWRITE_LONGER_CONTEXT_LEN=1 python -m sglang.launch_server --model-path Qwen/Qwen3-Next-80B-A
```

Nota

La variable de entorno `SGLANG_ALLOW_OVERWRITE_LONGER_CONTEXT_LEN=1` es necesaria en este momento.

Nota

La longitud de contexto predeterminada es de 256 KB. Considere reducir la longitud de contexto a un valor menor, por ejemplo, 32768, si el servidor no se inicia.

vLLM

[vLLM](#) es un motor de inferencia y servicio de alto rendimiento y bajo consumo de memoria para modelos LLM. vLLM puede utilizarse para lanzar un servidor con un servicio API compatible con OpenAI.

vLLM ha dado soporte a Qwen3-Next en su mainrama, que se puede instalar desde el código fuente:

caparazón

```
1 pip install vllm --pre --extra-index-url https://wheels.vllm.ai/nightly
```

El siguiente comando se puede utilizar para crear un punto final de API `http://localhost:8000/v1` con una longitud de contexto máxima de 256K tokens utilizando paralelismo tensorial en 4 GPU.

caparazón

```
1 VLLM_ALLOW_LONG_MAX_MODEL_LEN=1 vllm serve Qwen/Qwen3-Next-80B-A3B-Instruct --port 8000 --tensor-parallel-size 4 --m
```

Se recomienda el siguiente comando para MTP con el resto de la configuración igual que la anterior:

caparazón

```
1 VLLM_ALLOW_LONG_MAX_MODEL_LEN=1 vllm serve Qwen/Qwen3-Next-80B-A3B-Instruct --port 8000 --tensor-parallel-size 4 --m
```

Nota

La variable de entorno `VLLM_ALLOW_LONG_MAX_MODEL_LEN=1` es necesaria en este momento.

Nota

La longitud de contexto predeterminada es de 256 KB. Considere reducir la longitud de contexto a un valor menor, por ejemplo, 32768, si el servidor no se inicia.

Uso de agentes

Qwen3 destaca por sus capacidades de llamada a herramientas. Recomendamos usar [Qwen-Agent](#) para aprovechar al máximo la funcionalidad de Qwen3. Qwen-Agent encapsula internamente las plantillas y los analizadores de llamadas a herramientas, lo que reduce considerablemente la complejidad del código.

Para definir las herramientas disponibles, puede utilizar el archivo de configuración de MCP, la herramienta integrada de Qwen-Agent o integrar otras herramientas por su cuenta.

pitón

```
1 from qwen_agent.agents import Assistant
2
3 # Define LLM
4 llm_cfg = {
5     'model': 'Qwen3-Next-80B-A3B-Instruct',
6
7     # Use a custom endpoint compatible with OpenAI API:
8     'model_server': 'http://localhost:8000/v1', # api_base
9     'api_key': 'EMPTY',
10 }
11
12 # Define Tools
13 tools = [
14     {'mcpServers': { # You can specify the MCP configuration file
15         'time': {
16             'command': 'uvx',
17             'args': ['mcp-server-time', '--local-timezone=Asia/Shanghai']
18         },
19         "fetch": {
20             "command": "uvx",
21             "args": ["mcp-server-fetch"]
22         }
23     }
24 },
25     'code_interpreter', # Built-in tools
26 ]
27
28 # Define Agent
29 bot = Assistant(llm=llm_cfg, function_list=tools)
30
31 # Streaming generation
32 messages = [{'role': 'user', 'content': 'https://qwenlm.github.io/blog/ Introduce the latest developments of Qwen'}]
33 for responses in bot.run(messages=messages):
34     pass
```

```
print(responses)
```

Procesamiento de textos ultralargos

Qwen3-Next admite de forma nativa contextos de hasta 262 144 tokens. Para conversaciones cuya longitud total (incluyendo entrada y salida) supere significativamente este límite, recomendamos utilizar técnicas de escalado RoPE para gestionar textos largos de forma eficaz. Hemos validado el rendimiento del modelo con contextos de hasta 1 millón de tokens mediante el método [YaRN](#).

YaRN actualmente es compatible con varios marcos de inferencia, por ejemplo transformers, vllmy sglang. En general, existen dos enfoques para habilitar YaRN para los marcos compatibles:

Modificación de los archivos del modelo:

En el config.jsonarchivo, agregue los rope_scalingcampos:

json

```
1 {  
2     ...,  
3     "rope_scaling": {  
4         "rope_type": "yarn",  
5         "factor": 4.0,  
6         "original_max_position_embeddings": 262144  
7     }  
8 }
```

Pasar argumentos de línea de comandos: Para ello vllm, puede utilizar

caparazón

```
1 VLLM_ALLOW_LONG_MAX_MODEL_LEN=1 vllm serve ... --rope-scaling '{"rope_type":"yarn","factor":4.0,"original_max_position_er
```

Para ello sglang, puedes usar

caparazón

```
1 SGLANG_ALLOW_OVERWRITE_LONGER_CONTEXT_LEN=1 python -m sglang.launch_server ... --json-model-override-args '{"rope_
```

NOTA :

Todos los frameworks de código abierto más importantes implementan YaRN estático, lo que significa que el factor de escala permanece constante independientemente de la longitud de la entrada, **lo que podría afectar al rendimiento en textos más cortos.**

Recomendamos añadir la rope_scaling configuración únicamente cuando sea necesario procesar contextos largos.

También se recomienda modificarlo factor según sea necesario. Por ejemplo, si la longitud típica del contexto para su aplicación es de 524.288 tokens, sería mejor establecerlo factor en 2,0.

Resumen

Qwen3-Next representa un gran avance en la arquitectura del modelo, introduciendo innovaciones en los mecanismos de atención, incluyendo la atención lineal y la puerta de atención, así como una mayor dispersión en su diseño MoE. Qwen3-Next-80B-A3B ofrece un rendimiento a la par con el modelo superior Qwen3-235B-A22B-2507 tanto en modos de pensamiento como sin él, a la vez que proporciona una inferencia significativamente más rápida, especialmente en escenarios de contexto extenso. Con este lanzamiento, nuestro objetivo es empoderar a la comunidad de código abierto para que evolucione junto con los avances arquitectónicos de vanguardia. De cara al futuro, seguiremos perfeccionando esta arquitectura para desarrollar Qwen3.5, con el objetivo de alcanzar niveles de inteligencia y productividad sin precedentes.

Referencias

- 1 . Redes Delta con Puertas de Acceso: Mejora de Mamba2 con la regla Delta
- 2 . Atención controlada para modelos de lenguaje grandes: no linealidad, escasez y ausencia de sumidero de atención.

- 3 . Informe técnico de DeepSeek-V3
- 4 . Los detalles marcan la diferencia: Implementación de la función de pérdida de equilibrio de carga para el entrenamiento de modelos especializados de mezcla de expertos.
- 5 . Modelos de lenguaje de transmisión eficientes con sumideros de atención
- 6 . Activaciones masivas en modelos de lenguaje a gran escala
- 7 . Gemma: Modelos abiertos basados en la investigación y la tecnología de Gemini.
- 8 . Aproximación de redes de alimentación directa de dos capas para transformadores eficientes
- 9 . Mejores y más rápidos modelos de lenguaje a gran escala mediante predicción multi-token.
- 10 . ProphetNet: Predicción de N-gramas futuros para el preentrenamiento de secuencia a secuencia